DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

TRAVEL BOOKING DATA ORGANIZER

Representing Fixed Booking Details Using Tuples, Performing Immutability Operations, and Generating Aggregated Summaries in Python

Institution	**Budge Budge Institute of Technology, Kolkata**
Subject / Code	**Programming with Python / PCC-CS393**
Department & Session	Computer Science & Engineering | Academic Year 2026–2027
Curriculum Outcome	**CO4 & CO5** (Data Structures, Functions, and Tuple Manipulations)

GROUP MEMBERS & CREDENTIALS

Sl. No.	Candidate Name	University Roll Number
1	MD ALI ZINNA MOLLA	27600125093
2	ARGHA MAITY	27600125094
3	AKASHDIP SAH	27600125095
4	SHRUTI BISWAS	27600125096

1. ABSTRACT

The **Travel Booking Data Organizer** is a Python terminal application developed to manage, query, and analyze fixed passenger ticket reservations using immutable data structures. In commercial transport applications, core booking parameters—including Booking IDs, passenger names, travel destinations, journey dates, transport modes, and ticket fares—must remain write-protected post-confirmation. By utilizing Python **tuples**, this application enforces strict immutability for individual reservation records while supporting compound nested collections. The system features an interactive console menu to display detailed records, perform linear search lookups by Booking ID, extract and aggregate distinct travel routes using tuple comprehension, calculate cumulative revenue summaries, and demonstrate fundamental sequence behaviors (indexing, concatenation, repetition, and membership verification) using standard library components without external dependencies.

2. INTRODUCTION

Reliable information management is fundamental to passenger transport logistics. Confirmed reservations across railways, airlines, and intercity buses represent historical transactions that must resist unauthorized modification during runtime execution. Python's built-in `tuple` sequence type provides fixed memory allocation, reduced execution overhead, and native write protection compared to mutable lists. The *Travel Booking Data Organizer* was implemented to satisfy academic Course Outcomes CO4 and CO5, offering practical experience with functional data extraction, sequence slicing, linear search, and cumulative financial reporting.

3. PROBLEM STATEMENT

In introductory programming systems, transactional records are frequently stored in mutable structures such as lists or dictionaries. This architecture leaves confirmed ticket records vulnerable to accidental reassignments, faulty runtime overwrites, or unauthorized modifications during multi-step function processing. Furthermore, travel operators require straightforward console tools to verify ticket validity by ID, aggregate destination manifests, and compute financial metrics without endangering the underlying transaction records. A reliable, immutable, menu-driven data organizer is required to address these issues.

4. OBJECTIVES

- **Enforce Write-Once Integrity:** Store confirmed booking records as 6-element immutable tuples `(id, name, destination, date, transport, price)` to eliminate runtime corruption.
- **Organize Nested Structures:** Group atomic records into a composite collection tuple for unified sequence management.
- **Implement Target Search:** Build linear search algorithms to locate and format individual booking records by Booking ID.
- **Aggregate Financial Summaries:** Iterate across the booking collection to compute cumulative fare totals.
- **Extract Destination Manifests:** Employ tuple comprehension to extract and package journey destinations into lightweight sequence collections.
- **Demonstrate Tuple Operations:** Illustrate zero-based indexing, nesting, concatenation, repetition factors, and membership validation.

5. EXISTING VS. PROPOSED SYSTEM

Existing System (List-Based / Manual)	Proposed System (Tuple-Based Organizer)
Mutable storage allows accidental modification of fares or passenger names.	Tuples guarantee strict write-once immutability for confirmed booking records.
Higher memory overhead due to dynamic resizing buffers in lists.	Tuples provide optimized memory allocation and faster sequence access.
Search and filtering logic often lack structured validation.	Modular functions handle record display, search, and summation cleanly.
Manual calculations are vulnerable to human error.	Automated computation routines instantly summarize cumulative revenue.

6. SCOPE OF THE PROJECT

In-Scope: Immutable storage of 6-field booking tuples; nested sequence management; ID lookups with feedback; destination compilation via comprehension; cumulative revenue summation; sequence operations demonstration; interactive loop control.
Out-of-Scope: External database integration (SQL), graphical UI development (Tkinter), or online payment processing, which are identified as future enhancements.

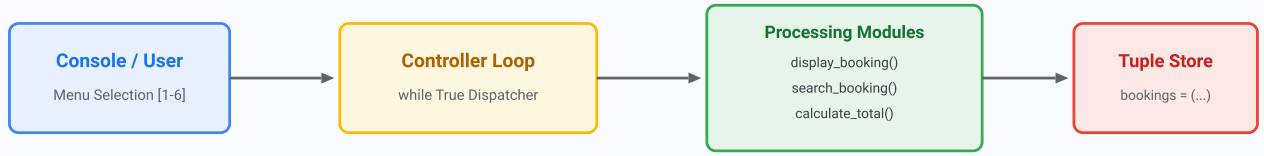
7. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Environment	PC / Laptop with Intel Core i3 / AMD Ryzen 3 or higher, minimum 2 GB RAM, 200 MB disk space
Operating System	Cross-platform: Microsoft Windows 10/11, Ubuntu Linux 20.04+, or macOS
Language & Environment	Python 3.8 to 3.12+ (CPython Standard Runtime)
Libraries & Packages	Python Standard Library (Pure built-in functions: `tuple()`, `len()`, `print()`, `input()`)
Development IDE	Visual Studio Code, PyCharm, IDLE, or standard terminal text editors

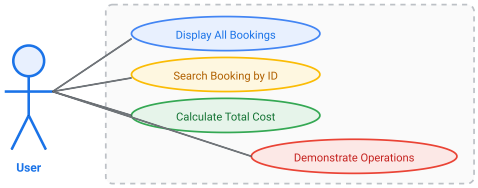
8. SYSTEM DESIGN & ARCHITECTURAL DIAGRAMS

The system utilizes a procedural architecture where the interactive driver dispatches user choices to modular processing functions, operating on immutable tuple collections.

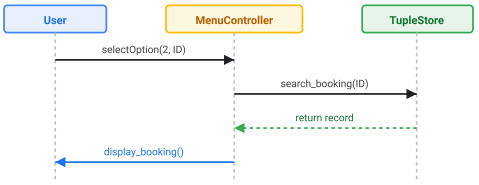
System Component & Data Flow Architecture:



UML Use Case Diagram



UML Sequence Diagram



9. MODULES DESCRIPTION

1. **Display Module (`display_booking`):** Extracts attributes by index (0=ID, 1=Passenger, 2=Destination, 3=Date, 4=Transport, 5=Fare) and renders formatted manifests.
2. **Revenue Computation Module (`calculate_total`):** Traverses the collection, accumulates index 5, and computes aggregate passenger expenditure.
3. **Destination Aggregation Module (`show_destinations`):** Uses tuple comprehension `tuple(b[2] for b in bookings)` to create an immutable route sequence and prints length via `len()`.
4. **Search Module (`search_booking`):** Performs linear lookup comparing target ID against index 0, displaying details if matched or reporting absence.
5. **Operations Demonstration Module:** Illustrates indexing (`[0]`), nested access (`[0][1]`), repetition (`* 2`), and membership testing (`in`).
6. **Driver & Control Module:** Manages the interactive `while True` loop, menu I/O, and clean program exit.

10. ALGORITHM AND FLOWCHART

System Flowchart:



Step-by-Step Program Logic:

1. **Initialization:** Define `booking1`, `booking2`, and `booking3` as 6-tuples. Bundle into compound tuple `bookings = (booking1, booking2, booking3)`.
2. **Menu Loop:** Present the interactive menu with options 1 through 6. Read user choice as a string.
3. **Option 1 (Display All):** Loop over `bookings`; pass each record to `display_booking()` to print formatted ticket details.
4. **Option 2 (Search by ID):** Prompt for integer `booking_id`. Traverse records linearly. If index 0 matches, invoke `display_booking()`; if loop concludes without a match, print "Booking not found!".
5. **Option 3 (Show Destinations):** Generate `destinations = tuple(b[2] for b in bookings)`. Print the tuple and `len(bookings)`.
6. **Option 4 (Calculate Total):** Call `calculate_total(bookings)` to sum index 5 across all records. Print total cost.
7. **Option 5 (Tuple Operations):** Demonstrate first element indexing (`bookings[0]`), nested access (`bookings[0][1]`), length calculation, composite representation, tuple repetition (`booking1 * 2`), and membership checking (`booking1 in bookings`).
8. **Option 6 (Exit):** Print confirmation message and terminate loop execution.
9. **Fallback:** For invalid selections, notify user and redisplay the menu.

11. IMPLEMENTATION ARCHITECTURE

The application is structured into a single modular Python script (`main.py`) using pure standard-library features. Immutability guarantees that reservation data remains write-protected across all operations. The complete, unedited source code is documented in **Appendix A (Page 7)**, followed by the complete execution trace in **Appendix B (Page 8)**.

12. TEST CASES, EXPECTED AND ACTUAL RESULTS

Test ID	Test Scenario / Input	Expected Result	Actual Result	Status
TC-01	Menu Choice 1 (Display All)	Display 3 formatted ticket summaries	All 3 records formatted	**PASS**
TC-02	Menu Choice 2 → ID: 102	Display ticket details for Priya	Priya / Delhi displayed	**PASS**
TC-03	Menu Choice 2 → ID: 999	"Booking not found!" notification	"Booking not found!"	**PASS**
TC-04	Menu Choice 3 (Destinations)	`('Kolkata', 'Delhi', 'Darjeeling')`	Tuple printed, count = 3	**PASS**
TC-05	Menu Choice 4 (Total Cost)	Total Cost: Rs. 6000 (850+4500+650)	Total: Rs. 6000	**PASS**
TC-06	Menu Choice 5 (Tuple Ops)	Indexing, Repetition, Membership	All operations verified	**PASS**
TC-07	Menu Choice 6 (Exit)	Thank you message & termination	Loop terminated	**PASS**

13. EXECUTION TRACE SAMPLE

The terminal window below highlights verification traces for ID search (Option 2) and total cost computation (Option 4):

Terminal — Search & Aggregation Verification

```
===== Travel Booking Data Organizer =====
1. Display All Bookings
2. Search Booking by ID
3. Show All Destinations
4. Calculate Total Booking Cost
5. Demonstrate Tuple Operations
6. Exit
Enter your choice: 2
Enter Booking ID: 102

--- Travel Booking Details ---
Booking ID: 102
Passenger Name: Priya
Destination: Delhi
Travel Date: 2026-11-10
Transport: Flight
Ticket Price: Rs. 4500

===== Travel Booking Data Organizer =====
1. Display All Bookings
2. Search Booking by ID
3. Show All Destinations
4. Calculate Total Booking Cost
5. Demonstrate Tuple Operations
6. Exit
Enter your choice: 4
Total Booking Cost: Rs. 6000
```

14. LIMITATIONS

- **In-Memory Volatility:** Records are stored in volatile RAM; confirmed bookings do not persist after the program exits.
- **Fixed Schema Dependency:** Relies on a strict 6-element index structure; missing fields will cause index errors.
- **Linear Search Complexity:** Uses O(n) search, which is optimal for small to medium lists but slower for massive enterprise datasets.
- **Console-Only Interface:** Lacks a graphical interface and requires integer inputs for ID queries.

15. CONCLUSION

The **Travel Booking Data Organizer** satisfies the requirements established under academic Course Outcomes CO4 and CO5. The application confirms that Python tuples provide an effective, lightweight mechanism for immutable record management in transactional environments. Confirmed bookings remain protected against accidental modification while enabling flexible data querying, route extraction, and revenue aggregation.

16. FUTURE SCOPE

- **File Persistence:** Implement JSON or CSV serialization to maintain bookings across sessions.
- **Dynamic Record Ingestion:** Add interactive prompts to append new immutable booking tuples via tuple concatenation (`bookings + (new_booking,)`).
- **Input Validation:** Add `try...except` exception handling around inputs to prevent crashes from non-integer values.
- **GUI Interface:** Develop a Tkinter or web dashboard to display records in interactive data tables.

17. REFERENCES

1. Python Software Foundation. (2024). *Built-in Types: Tuples and Sequence Types*. Python 3 Documentation. [Ref #1]
2. Mark Lutz. (2013). *Learning Python: Powerful Object-Oriented Programming (5th Edition)*. O'Reilly Media. ISBN: 978-1449355739. [Ref #2]
3. Guido van Rossum. (2024). *The Python Tutorial: Data Structures*. Official Python Documentation. [Ref #3]
4. Department of Computer Science & Engineering, Budge Budge Institute of Technology. (2026). *Lab Manual for IT Workshop (Python) PCC-CS393*. [Ref #4]

APPENDIX A: COMPLETE IMPLEMENTATION SOURCE CODE (MAIN.PY)

Complete, unclipped Python source code for the Travel Booking Data Organizer:

main.py — 100% Complete Source Code Listing

```python
def display_booking(booking):
    print("\n--- Travel Booking Details ---")
    print("Booking ID:", booking[0])
    print("Passenger Name:", booking[1])
    print("Destination:", booking[2])
    print("Travel Date:", booking[3])
    print("Transport:", booking[4])
    print("Ticket Price: Rs.", booking[5])

def calculate_total(bookings):
    total = 0
    for booking in bookings:
        total += booking[5]
    return total

def show_destinations(bookings):
    destinations = tuple(booking[2] for booking in bookings)
    print("\nAll Destinations:", destinations)
    print("Number of Bookings:", len(bookings))

def search_booking(bookings, booking_id):
    for booking in bookings:
        if booking[0] == booking_id:
            display_booking(booking)
            return
    print("Booking not found!")

# Fixed booking details stored as tuples
booking1 = (101, "Rahul", "Kolkata", "2026-11-05", "Train", 850)
booking2 = (102, "Priya", "Delhi", "2026-11-10", "Flight", 4500)
booking3 = (103, "Amit", "Darjeeling", "2026-11-15", "Bus", 650)

# Combine tuples into a collection
bookings = (booking1, booking2, booking3)

while True:
    print("\n===== Travel Booking Data Organizer =====")
    print("1. Display All Bookings")
    print("2. Search Booking by ID")
    print("3. Show All Destinations")
    print("4. Calculate Total Booking Cost")
    print("5. Demonstrate Tuple Operations")
    print("6. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        for booking in bookings:
            display_booking(booking)
    elif choice == "2":
        booking_id = int(input("Enter Booking ID: "))
        search_booking(bookings, booking_id)
    elif choice == "3":
        show_destinations(bookings)
    elif choice == "4":
        print("Total Booking Cost: Rs.", calculate_total(bookings))
    elif choice == "5":
        print("First Booking:", bookings[0])
        print("First Passenger:", bookings[0][1])
        print("Total Records:", len(bookings))
        print("Combined Booking Tuples:", bookings)
        print("Repeated Tuple:", booking1 * 2)
        print("Is Booking 101 Present:", booking1 in bookings)
    elif choice == "6":
        print("Thank you for using Travel Booking Data Organizer!")
        break
    else:
        print("Invalid choice! Please try again.")
```

APPENDIX B: COMPLETE APPLICATION EXECUTION OUTPUT

Complete sequential console output demonstrating menu choices 1 through 6:

Python 3.10 Interactive Console — Complete Execution Run

```
===== Travel Booking Data Organizer =====
1. Display All Bookings       2. Search Booking by ID
3. Show All Destinations      4. Calculate Total Booking Cost
5. Demonstrate Tuple Ops      6. Exit
Enter your choice: 1

--- Travel Booking Details ---
Booking ID: 101 | Passenger: Rahul | Destination: Kolkata
Travel Date: 2026-11-05 | Transport: Train | Ticket Price: Rs. 850
--- Travel Booking Details ---
Booking ID: 102 | Passenger: Priya | Destination: Delhi
Travel Date: 2026-11-10 | Transport: Flight | Ticket Price: Rs. 4500
--- Travel Booking Details ---
Booking ID: 103 | Passenger: Amit | Destination: Darjeeling
Travel Date: 2026-11-15 | Transport: Bus | Ticket Price: Rs. 650

===== Travel Booking Data Organizer =====
Enter your choice: 2
Enter Booking ID: 102
--- Travel Booking Details ---
Booking ID: 102
Passenger Name: Priya
Destination: Delhi
Travel Date: 2026-11-10
Transport: Flight
Ticket Price: Rs. 4500

===== Travel Booking Data Organizer =====
Enter your choice: 3
All Destinations: ('Kolkata', 'Delhi', 'Darjeeling')
Number of Bookings: 3

===== Travel Booking Data Organizer =====
Enter your choice: 4
Total Booking Cost: Rs. 6000

===== Travel Booking Data Organizer =====
Enter your choice: 5
First Booking: (101, 'Rahul', 'Kolkata', '2026-11-05', 'Train', 850)
First Passenger: Rahul
Total Records: 3
Combined Booking Tuples:
  ((101, 'Rahul', 'Kolkata', '2026-11-05', 'Train', 850),
   (102, 'Priya', 'Delhi', '2026-11-10', 'Flight', 4500),
   (103, 'Amit', 'Darjeeling', '2026-11-15', 'Bus', 650))
Repeated Tuple:
  (101, 'Rahul', 'Kolkata', '2026-11-05', 'Train', 850,
   101, 'Rahul', 'Kolkata', '2026-11-05', 'Train', 850)
Is Booking 101 Present: True

===== Travel Booking Data Organizer =====
Enter your choice: 6
Thank you for using Travel Booking Data Organizer!
```